

Experiment 1

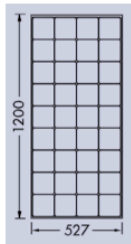
Stewart Rojec

The first step of the experiment was to determine the efficiencies under two different sets of test conditions. The irradiance given by the test condition is multiplied by the panel area to get a figure for incident power, and then the given panel rated power is divided by that figure to give efficiency. The calculation for STC is shown on the left, and NOCT on the right.

Step 1 Based on the outside dimensions and the maximum-power point (MPP) electrical characteristics of SQ-85 PV panels, calculate the panel solar-to-electrical efficiency under (i) standard test conditions (STC), and (ii) nominal operating cell temperature (NOCT) conditions.

Grading rubric for [Step 1](#):

PV panel efficiencies under STC and NOCT: [6 pts]



$$\begin{aligned}
 &1.2 \text{ m} \cdot 0.527 \text{ m} = 0.6324 \text{ m}^2 \\
 &0.6324 \cdot 1000 \frac{\text{W}}{\text{m}^2} = 632.4 \text{ W} \\
 &V_{\text{mpp}} = 17.2 \text{ V} \\
 &I_{\text{mpp}} = 4.95 \text{ A} \quad (\text{STC}) \\
 &17.2 \cdot 4.95 = 85.14 \text{ W} \\
 &\frac{85.14 \text{ W}}{632.4 \text{ W}} = 13.46\% \quad (\text{STC}) \\
 &0.6324 \cdot 800 \frac{\text{W}}{\text{m}^2} = 505.92 \text{ W} \\
 &P_{\text{mpp}} = 63 \text{ W} \quad (\text{NOCT}) \\
 &\frac{63}{505.92} = 12.45\% \quad (\text{NOCT})
 \end{aligned}$$

Figure 1: Efficiency Calculations

Step 2:

The purpose of step 2 was to determine more panel model parameters from the ones given in the datasheet. The MATLAB script to accomplish this takes the KCL equation from each of the circuit conditions (short circuit, open circuit, and maximum power point), as well as the derivative of the maximum power for the fourth equation and solves them symbolically in two steps. The code is shown in Figure 2 below.

```

Vmpp = 17.2;
Impp = 4.95;
Voc = 22.2;
Isc = 5.45;
Ns = 36;
Vt = .02569;
insol = 1000; %w/m2
syms Ido ki Rp Rs
%at open circuit
eqoc = ki*insol == (Ido*(exp(Voc/(Ns*Vt))-1)) + Voc/(Rp*Ns)
%at short circuit
eqsc = ki*insol == (Ido*(exp(((Isc*Rs)/Vt)) - 1)) + Isc*Rs/Rp + Isc
[k,i,Ido] = vpasolve([eqoc, eqsc],[ki,Ido])
%at max power
Vdmp = (Vmpp/Ns) + Impp*Rs;
eqmp = ki*insol - Ido*(exp(Vdmp/Vt)-1) - (Vdmp/Rp) == Impp; %max power
eqpr = 0 == ((-Ido*Vmpp)/(Ns*Vt))*exp(Vdmp/Vt) - Vmpp/(Ns*Rp) + Impp % dp/dv

[Rp1,Rs1] = vpasolve([eqmp,eqpr],[Rp,Rs])
ki = subs(subs(ki,Rs,Rs1),Rp,Rp1)
Ido = subs(subs(Ido,Rs,Rs1),Rp,Rp1)

```

Figure 2: Circuit Parameter Script

The open circuit, short circuit, maximum power, and derivative equations are shown above as “eqoc,” “eqsc,” “eqmp,” and “eqpr,” respectively. The derivative of maximum power is then set to zero, since the power graph has a slope of zero at the maximum power point. The results are shown below.

```

Rp1 = 1.9068082756905244622770914093543
Rs1 = 0.012400781236233225156603650632674
ki = 0.0054854436591066373270720255780736
Ido = 0.00000000019406654402186378922321661291722

```

Figure 3: Circuit Parameter Results

Step 3:

The purpose of Step 3 was to integrate the circuit parameters we just derived from the MATLAB script into an LTSpice circuit model of the panel to simulate its behavior. The schematic is shown in Figure 4 below.

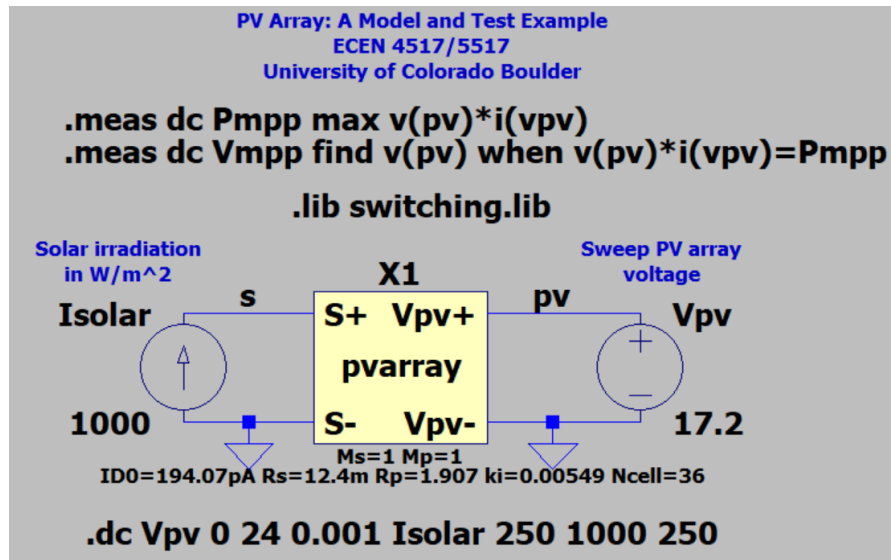


Figure 4: PV Array LTSpice Model

The spice directive at the bottom of Figure 4 sweeps the panel voltage from 0-24V and the insolation from 250 to 1000 watts per square meter. Below, the graphs of the maximum power points are shown.

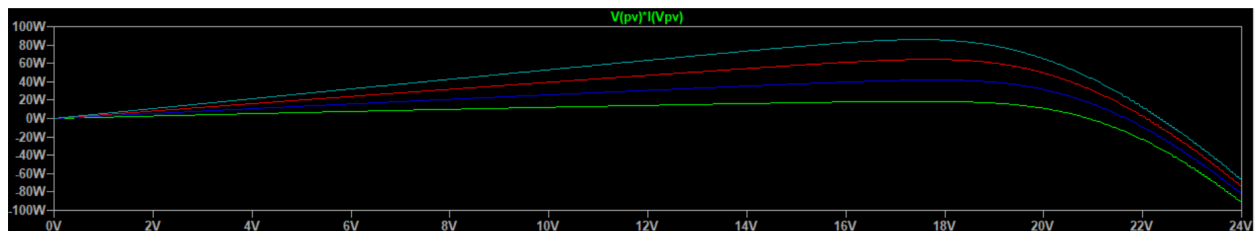


Figure 5: IV Characteristic at 4 Irradiances

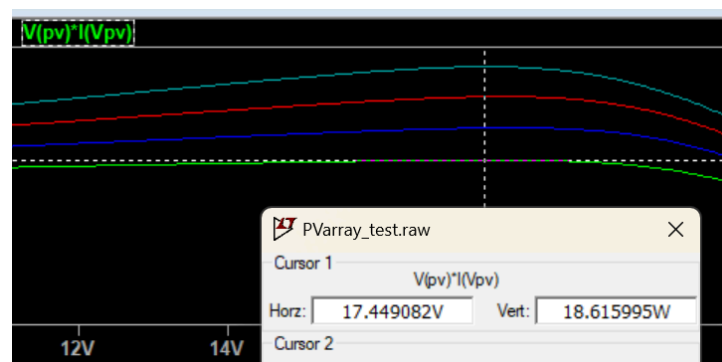


Figure 6: MPP at 250 W/m²

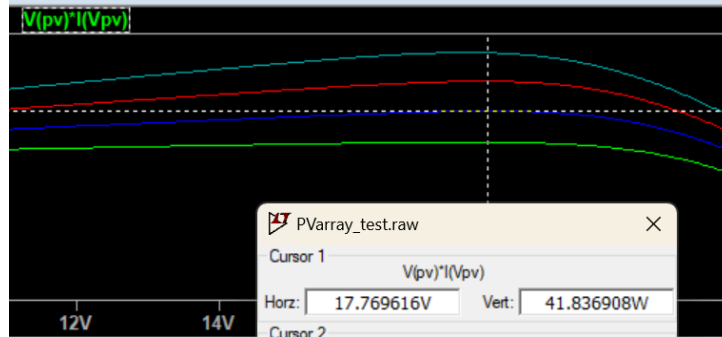


Figure 7: MPP at 500 W/m²

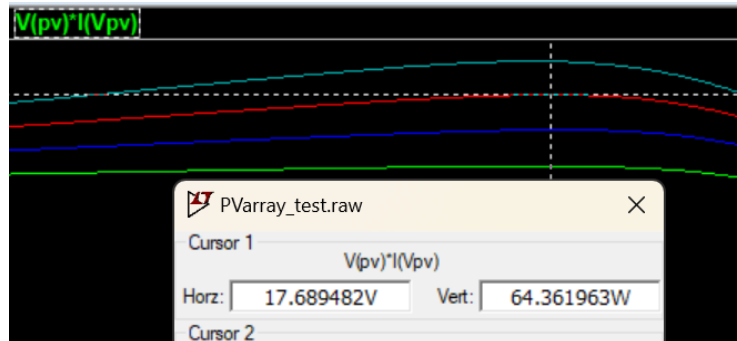


Figure 8: MPP at 750 W/m²

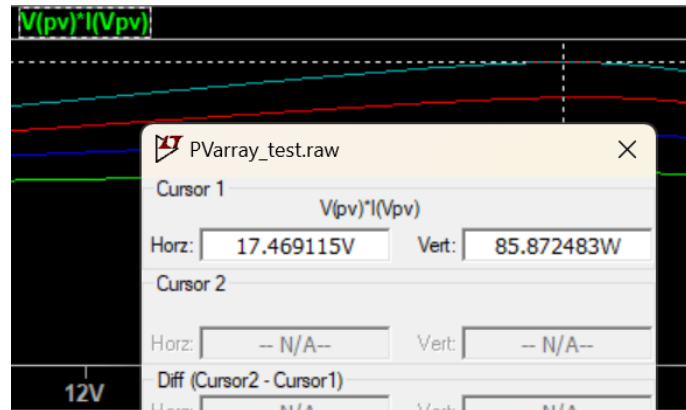


Figure 9: MPP at 1000 W/m²

Irradiance (W/m ²)	250	500	750	1000
Pmpp (W)	18.62	41.84	64.36	85.87
Vmpp (V)	17.45	17.77	17.69	17.47
Impp (A)	1.07	2.35	3.64	4.92

Table 1: Vmpp and Impp Results

Step 4:

The purpose of Step 4 was to use LTSpice to simulate using the PV panel to charge a battery, so the panel voltage is set to 12.55 to represent the battery at 75% SOC. The schematic is shown below.

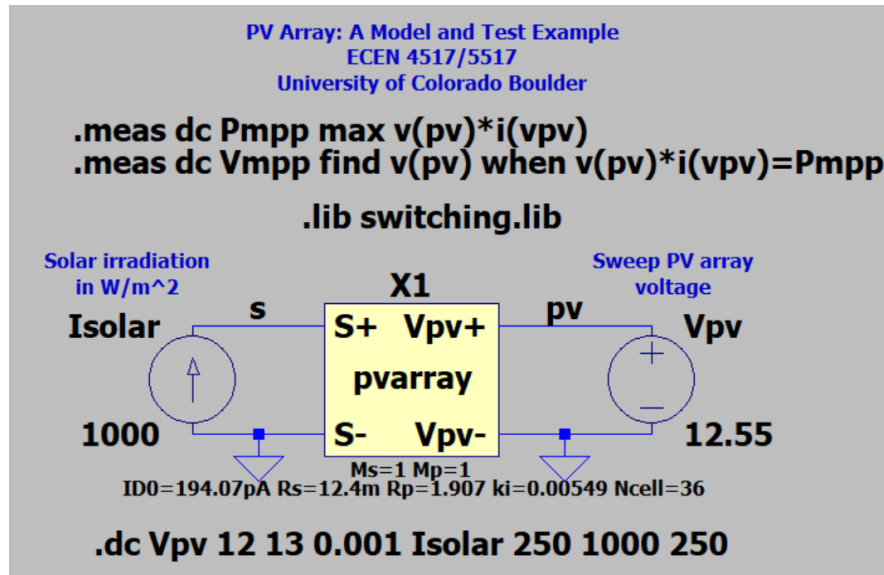


Figure 10: DET Schematic

The spice directive is changed to sweep from 12 to 13 V to make it easier to isolate the target battery operating point of 12.55 V. The power and current to the battery at this voltage and each irradiance is shown below.

Irradiance (W/m ²)	250	500	750	1000
Pbatt (W)	14.83	31.94	49.05	66.16
Ibatt (A)	1.18	2.54	3.91	5.27

Table 2: DET Battery Power Results

Step 5:

The purpose of Step 5 was to model and design a buck converter to allow the battery to be charged at the panel's maximum power point. To determine the inductance of the buck converter, the inductor current was calculated from the battery voltage and panel power, and then an inductance derived from the given current ripple. The calculations are shown below.

$$\begin{aligned} I_L &= \frac{85 \text{ W}}{12.55 \text{ V}} \\ &= 6.773 \text{ A} \\ \Delta I_L &= 6.773 \cdot 20\% \\ &= 1.35 \text{ A} \end{aligned}$$
$$\begin{aligned} 2\Delta I_L &= \frac{V_g}{L} D T_s \\ L \frac{di}{dt} &= V_g D T_s \\ L &= \frac{V_g D T_s}{2\Delta I_L} \\ L &= 46 \mu\text{H} \end{aligned}$$

Figure 11: Inductance Calculation

The duty cycle and corresponding control voltage was determined from the ideal V_{out}/V_{in} ratio of a buck converter, using the maximum power voltage and the battery voltage as V_{in} and V_{out} , respectively. The final schematic for the buck converter circuit is shown below.

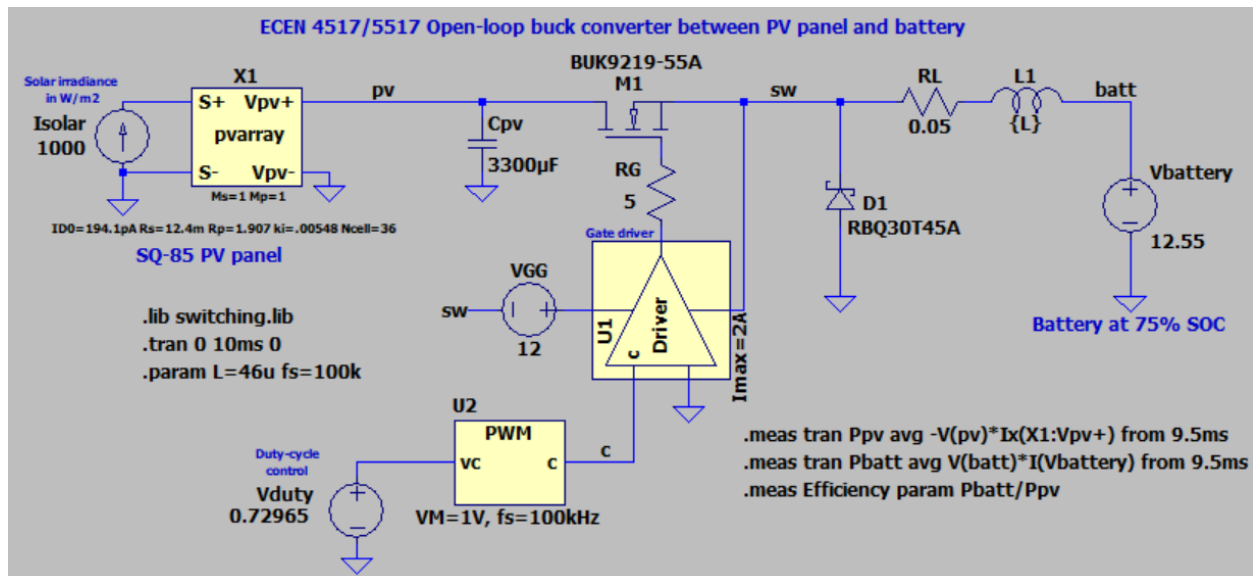


Figure 12: Buck Converter Schematic

Several simulation results from the circuit in Figure 12 are shown below.

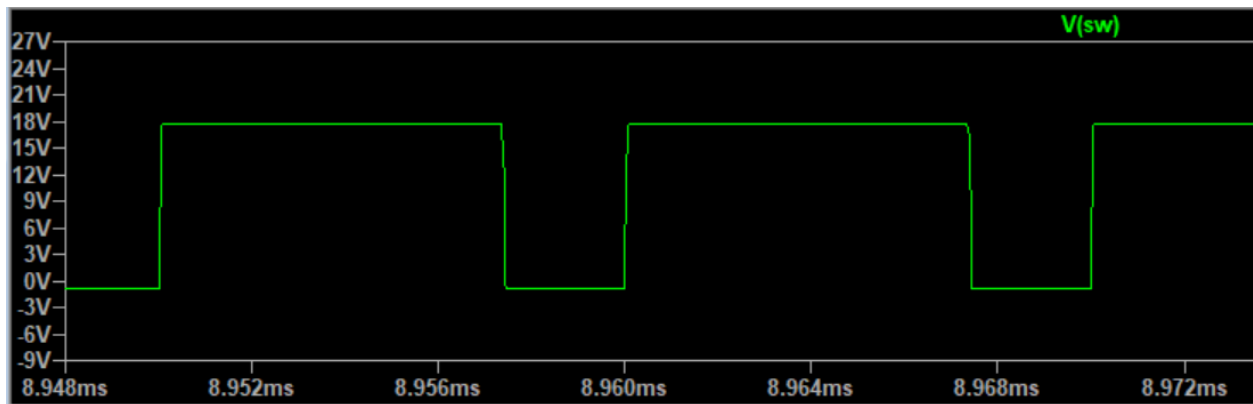


Figure 13: Switched-Node Voltage (V_{sw})

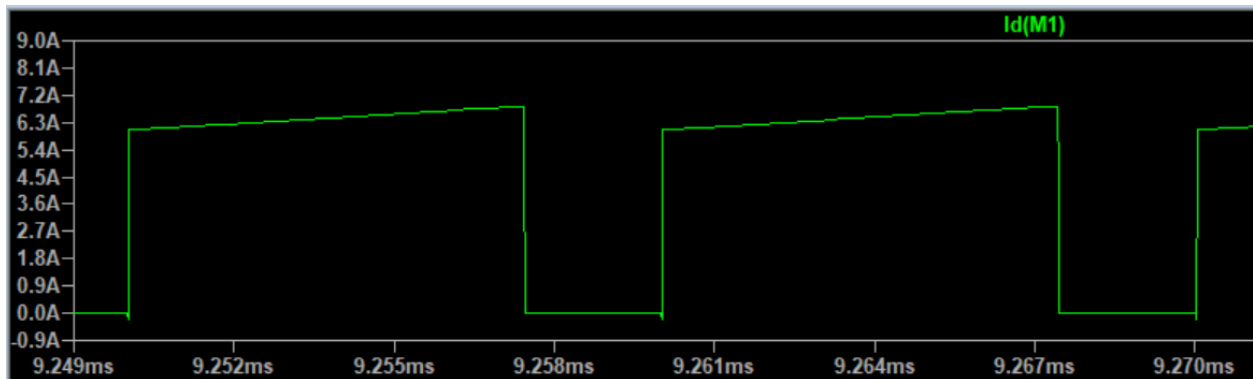


Figure 14: Transistor Drain Current

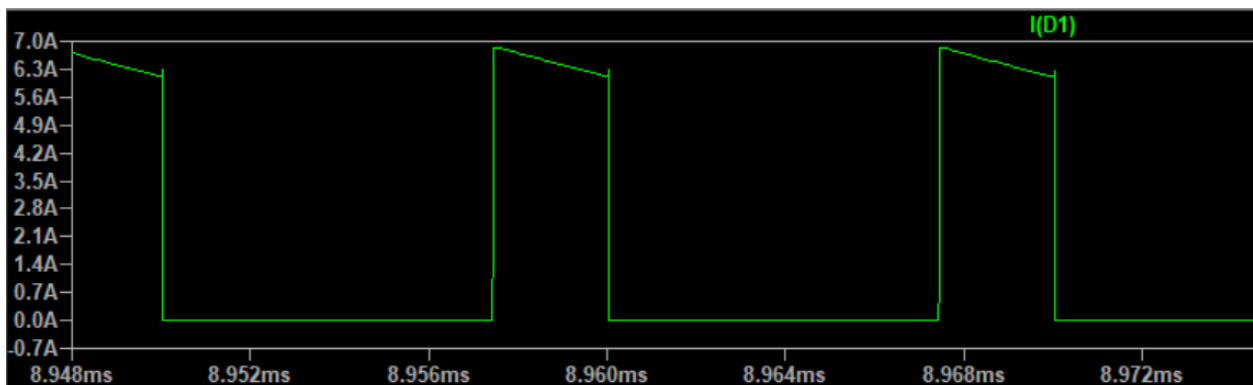


Figure 15: Diode Current

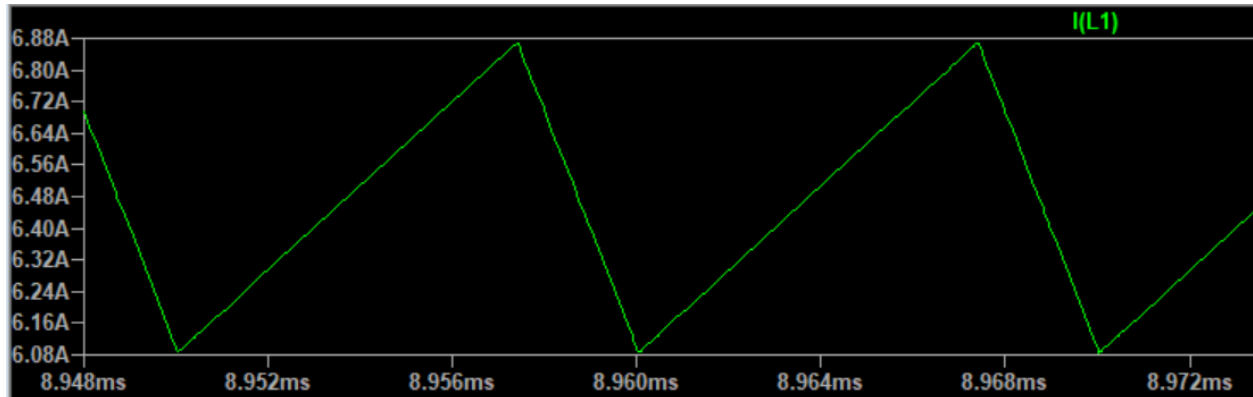


Figure 15: Inductor Current

The RMS values are taken from the steady state of several current figures using the LTSpice averaging function, shown below.

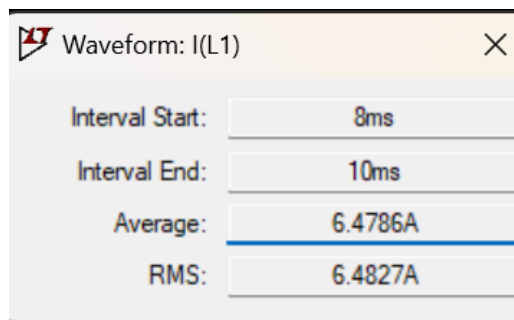


Figure 16: RMS Inductor Current

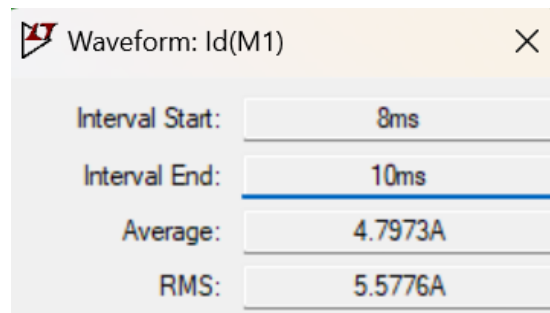


Figure 17: RMS Transistor Current

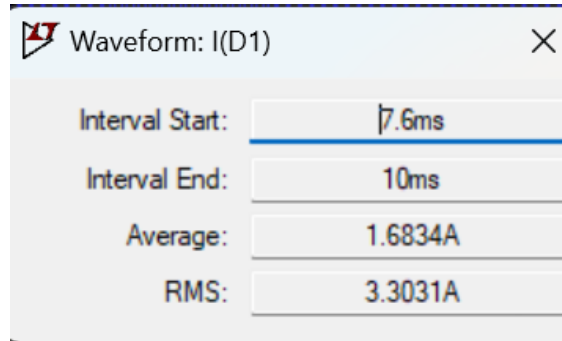


Figure 18: RMS Diode Current

The purpose of Step 8 was to familiarize with the IDE used for programming our development board (installing said IDE was Step 7). The given PWM_ADC file created a square wave on pin J5.3 on the board which we used the given values from the file and measured expressions to determine the switching frequency and the pin voltage. The calculations are shown below.

Step 8 Follow the steps in the slides, “Setting Up the PWM_ADC Project in CCS” to load the PWM_ADC project, open PWM_ADC.c, build code, load onto the LaunchPad target processor and run the code. With CCS in the debugger mode, add `sensed_voltage`, `pulse_width`, and `period` as expressions to monitor. You may need to refresh the expressions before recording the monitored values. With the parameters as provided in the code, what are the PWM frequency f_s and the duty cycle D of the PWM output waveform on pin J6.1 (pin 1 of the pin-header J6 on the LaunchPad)? Based on the `sensed_voltage` value, what is the voltage V_s at the ADC input pin J5.3?

$$\text{Duty cycle} = \text{pulse width} / \text{period} = 100/500 = 20\%$$

$$\frac{2^{12} - 1}{3.3} = \frac{2704}{V_{J5.3}}$$

$$V_{J5.3} = 2.18 \text{ V}$$

$$\text{period} = \# \text{ of clock cycles} @ 60 \text{ MHz}$$

$$f_s = \frac{60 \text{ MHz}}{500} = 120 \text{ KHz}$$

Figure 19: Step 8 Calculations

The “`sensed_voltage`” figure was 2704, which is converted to voltage using the ratio of maximum hexadecimal value to the maximum voltage sensed. The frequency is calculated using the board’s clock frequency, which is 60 MHz.

Step 9:

The purpose of Step 9 was to modify the given file to control the duty cycle and switching frequency. The modified code is shown below.

```

64 period = 600; // PWM period set as the number of (1/60 MHz) clock cycles
65 InitEPwm1Example(period, 0); // PWM setup with pulse width set to zero
66
67 EALLOW;
68 SysCtrlRegs.PCLKCR0.bit.TBCLKSYNC = 1;
69 EDIS;
70
71 // Step 5. Example specific code, enable interrupts:
72
73 // Enable CPU INT3 which is connected to EPWM1-3 INT:
74 PieCtrlRegs.PIEIER1.bit.INTx1 = 1; // Enable INT 1.1 in the PIE
75 IER |= M_INT1; // Enable CPU Interrupt 1
76
77 // Enable global Interrupts and higher priority real-time debug events:
78 EINT; // Enable Global interrupt INTM
79 ERTM; // Enable Global realtime interrupt DBGM
80
81 // ***** End of Setup *****
82
83
84 // User code
85
86 pulse_width = 450; // PWM duty cycle is D = pulse_width/period
87 update_duty(pulse_width); //Changes the duty cycle of the PWM output signal on pin J6.1
88

```

Figure 20: Modified PWM_ADC Code

The pulse_width and the period were both changed so that the clock frequency divided by the period was equal to the target frequency, and the pulse width was 75% of that period.

Step 10:

The goal of the last step was to see the effects of an RC filter on the ADC converter when filtering the PWM wave as an input. The RC filter had a bandwidth of roughly 1000 Hz using a 1.5k resistor and a 0.1uF capacitor. The same period as Figure 20 is used to create the PWM. The results of the filtering and ADC conversion is shown below.

Duty Cycle	voltage_sense	Measured Voltage (V)
10%	440	0.355
50%	2084	1.679
90%	3684	2.969

Table 3: ADC Conversion Results

This matches quite well with expectations, since the average voltage is higher for a higher duty cycle, so we expect the filtered voltage to also be higher.